



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Lenguajes y Compiladores

Análisis Sintáctico

(Informe de investigación)

Profesor:

Ing. Félix Márquez

Alumnos:

Victor Canchis V-27.935.276

Blas Gonzalez V-31.074.243

Diosdado Muñoz V-30.932.108

Celimar Rojas V-31.981.398

Ciudad Guayana, julio de 2026

Índice

Árboles de Sintaxis Abstracta (AST)	4
<i>Características principales de un AST</i>	<i>4</i>
<i>Implementación práctica en Python</i>	<i>6</i>
Comparación entre los análisis LL y LR	10
Análisis LL	10
<i>Conceptos Clave de LL</i>	<i>10</i>
<i>¿Cómo funciona un analizador LL?</i>	<i>10</i>
<i>Algoritmo de decisión:</i>	<i>11</i>
<i>Construcción de la Tabla:</i>	<i>11</i>
<i>Limitaciones de las Gramáticas LL (1)</i>	<i>12</i>
<i>Ventajas.....</i>	<i>12</i>
<i>Desventajas</i>	<i>13</i>
Analizador LR	13
<i>Descripción del analizador LR:</i>	<i>13</i>
<i>Algoritmo de análisis LR:</i>	<i>13</i>
<i>Tabla de análisis:.....</i>	<i>14</i>
<i>¿Cómo funciona el analizador LR?</i>	<i>14</i>
<i>Ventajas.....</i>	<i>15</i>
<i>Desventajas</i>	<i>15</i>

Resumen de los Generadores de Analizadores Sintáctico.....	16
<i>Generadores analizados.....</i>	16
<i>Flex</i>	<i>16</i>
<i>Bison</i>	<i>16</i>
<i>ANTLR.....</i>	<i>17</i>
<i>Gestión de errores</i>	17
<i>Bison: recuperación basada en el token error</i>	<i>18</i>
<i>ANTLR: estrategia de recuperación automática</i>	<i>18</i>
<i>Contraste.....</i>	<i>19</i>
Referencias.....	20

Árboles de Sintaxis Abstracta (AST)

Imagina que estás leyendo una oración en español. El árbol de sintaxis concreta sería como un análisis gramatical súper detallado que incluye hasta las comas y los puntos. El Árbol de Sintaxis Abstracta (AST) , en cambio, es como un resumen ejecutivo: se queda solo con lo esencial, con la estructura que realmente importa para entender el significado.

En el mundo de la programación, cuando escribes código, el compilador necesita entender qué estás diciendo. El AST es esa representación simplificada pero completa de tu programa, organizada en forma de árbol, donde:

- Las hojas son los valores más pequeños (números, nombres de variables, etc.)
- Los nodos internos son las operaciones o instrucciones (sumas, llamadas a funciones, condicionales, etc.)

Características principales de un AST

CARACTERÍSTICA	¿QUÉ SIGNIFICA EN PALABRAS SIMPLES?
ABSTRACCIÓN	<i>Se salta los detalles que no afectan el significado, como paréntesis, puntos y comas, o espacios en blanco.</i>
JERARQUÍA	<i>Organiza el código en una estructura de árbol que refleja cómo se agrupan las instrucciones.</i>
LIMPIEZA	<i>Es más compacto que el árbol de análisis concreto, eliminando nodos que solo existen por la gramática pero no aportan información semántica.</i>
REUTILIZABLE	<i>Una vez construido, sirve como base para las siguientes fases del compilador: análisis semántico, optimización y generación de código.</i>

Ejemplo 1: Expresión aritmética

Código de entrada:

```
x = 2 + 3 * 4
```

AST resultante:

```

=
/ \
x  +
   / \
   2  *
     / \
     3  4

```

El AST no incluye los detalles superfluos como el punto y coma, los espacios o los paréntesis (si los hubiera). Solo guarda la estructura esencial: una asignación (=) donde a la izquierda está la variable x y a la derecha una suma (+) que a su vez tiene una multiplicación (*).

Ejemplo 2: Estructura condicional

Código de entrada:

```

if (x > 5) {
    print("mayor")
} else {
    print("menor")
}

```

```
}
```

AST resultante:

```

    if
      / \
    >   print("mayor")
  / \
x    5
 \
  print("menor")

```

El AST captura la esencia: un nodo if que tiene una condición (una comparación > entre x y 5) y dos ramas (la instrucción que se ejecuta si se cumple y la que se ejecuta si no).

Implementación práctica en Python

Definimos los tipos de nodos que puede tener nuestro AST

```

class Nodo:

    pass

class Numero(Nodo):

    def __init__(self, valor):

        self.valor = valor

class Suma(Nodo):

    def __init__(self, izquierda, derecha):

```

```

        self.izquierda = izquierda

        self.derecha = derecha

class Multiplicacion(Nodo):

    def __init__(self, izquierda, derecha):

        self.izquierda = izquierda

        self.derecha = derecha

class Asignacion(Nodo):

    def __init__(self, variable, expresion):

        self.variable = variable

        self.expresion = expresion

```

Construimos el AST para: $x = 2 + 3 * 4$

```

ast = Asignacion(

    variable="x",

    expresion=Suma(

        izquierda=Numero(2),

        derecha=Multiplicacion(

            izquierda=Numero(3),

            derecha=Numero(4)

        )

    )

)

```

Función para "imprimir" el AST de forma legible

```
def mostrar_ast(nodo, nivel=0):

    indent = "  " * nivel

    if isinstance(nodo, Numero):

        print(f"{indent}Numero({nodo.valor})")

    elif isinstance(nodo, Suma):

        print(f"{indent}Suma")

        mostrar_ast(nodo.izquierda, nivel+1)

        mostrar_ast(nodo.derecha, nivel+1)

    elif isinstance(nodo, Multiplicacion):

        print(f"{indent}Multiplicacion")

        mostrar_ast(nodo.izquierda, nivel+1)

        mostrar_ast(nodo.derecha, nivel+1)

    elif isinstance(nodo, Asignacion):

        print(f"{indent}Asignacion -> {nodo.variable}")

        mostrar_ast(nodo.expresion, nivel+1)
```

```
mostrar_ast(ast)
```

Salida:

```
text
```

```
Asignacion -> x
```

```
Suma
```


Numero (2)

Multiplicacion

Numero (3)

Numero (4)

Este ejemplo muestra cómo se puede representar la estructura del código de manera jerárquica y legible para la máquina.

Comparación entre los análisis LL y LR

Análisis LL

El análisis LL es un método de análisis sintáctico descendente (top-down parser) que procesa el código fuente de izquierda a derecha construyendo una derivación por la izquierda LL parser. Se utiliza en la fase de análisis sintáctico de un compilador para verificar que el código cumple con las reglas gramaticales del lenguaje de programación.

Conceptos Clave de LL

El análisis se denota como LL(k), donde cada letra y variable tiene un significado operativo:

- **Primera L:** Indica que la entrada se lee de izquierda a derecha (Left-to-right).
- **Segunda L:** Indica que se construye una derivación por la izquierda (Leftmost derivation).
- **k:** Representa el número de tokens de anticipación (lookahead). Es el número de símbolos futuros que el analizador examina para decidir qué regla gramatical aplicar sin equivocarse. El más común y eficiente es LL (1).

¿Cómo funciona un analizador LL?

El algoritmo es determinista y no requiere volver atrás (backtracking). Utiliza tres elementos principales:

- **La Cadena de Entrada:** El flujo de tokens generados por el analizador léxico.
Termina con un delimitador especial.

- **La Pila (Stack):** Almacena los símbolos de la gramática. Se inicializa con el símbolo inicial de la gramática sobre el delimitador.
- **La Tabla de Análisis:** Una matriz bidimensional donde las filas son símbolos no terminales, las columnas son símbolos terminales, y las celdas contienen la regla de producción que se debe aplicar

Algoritmo de decisión:

- Si el tope de la pila es un terminal y coincide con el token actual de la entrada, se eliminan ambos y se avanza en la lectura.
- Si el tope de la pila es un no terminal A y el token actual es a , el analizador busca en la celda $M[A, a]$ de la tabla. Reemplaza A en la pila por el cuerpo de la regla de producción correspondiente.
- Si hay coincidencia con $\$,$ el análisis es exitoso. Si la celda está vacía, se reporta un error sintáctico.

Construcción de la Tabla:

Para construir la tabla de un analizador LL (1), se deben calcular matemáticamente dos funciones sobre la gramática:

- **FIRST(α):** El conjunto de terminales que pueden aparecer al inicio de las cadenas derivadas de la secuencia de símbolos α . Si α puede derivar en la cadena vacía (ϵ), entonces $\epsilon \in \text{FIRST}(\alpha)$.

- **FOLLOW(A):** El conjunto de terminales que pueden aparecer inmediatamente a la derecha del no terminal A en alguna forma sentencial. Siempre incluye el símbolo \$ para el símbolo inicial de la gramática.

Limitaciones de las Gramáticas LL (1)

No todas las gramáticas libres de contexto pueden ser analizadas por un método LL(1). Para que una gramática sea LL(1), su tabla de análisis no debe tener conflictos (más de una regla en la misma celda). Esto exige que la gramática cumpla las siguientes condiciones:

- No ser ambigua: Una cadena no puede tener dos árboles de análisis válidos.
- No tener recursividad por la izquierda: Las reglas de la forma $A \rightarrow A\alpha$ causan bucles infinitos en algoritmos descendentes. Deben eliminarse algebraicamente.
- Estar factorizada por la izquierda: el analizador no sabrá cuál elegir basándose solo en el primer token.

Ventajas

- **Simplicidad:** Es fácil de entender, diseñar e implementar de forma manual mediante funciones recursivas.
- **Depuración:** Los errores sintácticos se detectan y reportan de manera más rápida y cercana al punto donde ocurren, facilitando mensajes de error amigables para el usuario.
- **Eficiencia:** Para gramáticas compatibles, el proceso es directo y rápido sin consumir excesiva memoria.

Desventajas

- **Restricciones severas:** Obliga a transformar la gramática eliminando la recursividad por la izquierda y aplicando factorización por la izquierda.
- **Menor capacidad:** No puede parsear gramáticas complejas o ambiguas que un lenguaje moderno a veces requiere de forma natural.

Analizador LR

El analizador LR es un analizador ascendente para gramáticas libres de contexto, ampliamente utilizado por compiladores de lenguajes de programación y otras herramientas relacionadas. El analizador LR lee la entrada de izquierda a derecha y produce una derivación de extremo a extremo. Se denomina analizador ascendente porque intenta reducir las producciones gramaticales de nivel superior construyendo desde las hojas. En la práctica, los analizadores LR son los más potentes entre todos los analizadores deterministas.

Descripción del analizador LR:

El término analizador LR(k) se refiere a la exploración de izquierda a derecha, R a la derivación más a la derecha en sentido inverso y k al número de símbolos de entrada de "adelanto" no consumidos que se utilizan para tomar decisiones del analizador.

Algoritmo de análisis LR:

El algoritmo de análisis LR es el mismo para todos los analizadores, pero la tabla de análisis es diferente para cada uno. Consta de los siguientes componentes.

1. **Búfer de entrada:** contiene la cadena de texto proporcionada y termina con el símbolo \$.

2. **Pila:** La combinación del símbolo de estado y el símbolo de entrada actual se utiliza para hacer referencia a la tabla de análisis sintáctico con el fin de tomar las decisiones de análisis.

Tabla de análisis:

La tabla de análisis se divide en dos partes: tabla de acciones y tabla de destino. La tabla de acciones proporciona una regla gramatical para implementar el estado actual y el terminal actual en el flujo de entrada. Hay cuatro casos que se utilizan en la tabla de acciones, como se muestra a continuación.

- **Acción de desplazamiento:** en la acción de desplazamiento, el terminal actual se elimina del flujo de entrada y el estado n se inserta en la pila, convirtiéndose en el nuevo estado actual.
- **Acción de reducción:** El número m se escribe en el flujo de salida.
- El símbolo m mencionado en el lado izquierdo de la regla m indica que el estado se elimina de la pila.
- El símbolo m mencionado en el lado izquierdo de la regla m indica que se busca un nuevo estado en la tabla goto y se convierte en el nuevo estado actual agregándolo a la pila.

¿Cómo funciona el analizador LR?

El proceso opera con una pila (stack) y una tabla de análisis impulsada por un autómata finito. En cada paso, el analizador revisa la parte superior de la pila y el símbolo actual de entrada para ejecutar una de estas acciones básicas:

- **Desplazamiento (Shift):** Lee el siguiente token de la entrada y lo empuja a la pila.

- **Reducción (Reduce):** Cuando los elementos en la cima de la pila coinciden con el cuerpo de una regla gramatical (lado derecho), los reemplaza por el símbolo no terminal correspondiente (lado izquierdo).
- **Aceptación (Accept):** El análisis finaliza con éxito al validar toda la cadena.
- **Error:** Se activa si la combinación de la pila y la entrada no tiene una acción válida en la tabla

Ventajas

- **Gran alcance:** Reconoce casi todas las gramáticas libres de contexto deterministas que se usan en lenguajes de programación reales.
- **Detección temprana de errores:** Identifica fallos de sintaxis en el momento exacto en que la cadena deja de coincidir con la gramática.
- **Eficiencia:** Analiza el texto de entrada en tiempo lineal.

Desventajas

- **Complejidad:** Las tablas de estados y transiciones son gigantescas y difíciles de auditar o corregir si la gramática tiene conflictos (como *Shift-Reduce* o *Reduce-Reduce*).
- **Manejo de errores:** Detectar la posición exacta del error y recuperarse de él es más difícil debido a la naturaleza ascendente del algoritmo.
- **No apto para humanos:** Intentar programar un parser LR desde cero sin un generador automático es impráctico y propenso a errores graves.

Resumen de los Generadores de Analizadores Sintáctico

Un generador de analizadores sintácticos toma una gramática libre de contexto anotada y produce automáticamente el código fuente de un parser, evitando su codificación manual. Flex y Bison cubren el enfoque clásico basado en autómatas y tablas LALR para C/C++; ANTLR cubre un enfoque más moderno, multi-lenguaje, orientado a la construcción directa del árbol de sintaxis. Los tres difieren no solo en el algoritmo de análisis que implementan, sino también en cómo gestionan los errores: Bison exige que el desarrollador declare explícitamente las reglas de recuperación en la gramática, mientras que ANTLR incorpora una estrategia de recuperación automática que solo se sobrescribe si se necesita un comportamiento distinto.

Generadores analizados

Flex

Flex genera analizadores léxicos a partir de un conjunto de expresiones regulares asociadas a acciones en C. Internamente construye un autómata finito determinista que reconoce los patrones definidos y produce el flujo de tokens que alimenta al parser. No participa en el análisis sintáctico; su función se limita a la fase léxica, previa a Bison o a cualquier parser que consuma sus tokens.

Bison

Bison es la herramienta compatible con Yacc más usada para C/C++. Convierte una gramática libre de contexto anotada en un parser LALR(1), un subconjunto de LR que colapsa estados con el mismo núcleo para reducir el tamaño de las tablas de análisis, a costa

de perder algo de precisión frente a un LR(1) completo. Desde la versión 2.x, Bison también admite generar parsers GLR (Generalized LR), pensados para gramáticas ambiguas o no deterministas que un LALR(1) puro no puede procesar.

ANTLR

ANTLR (ANother Tool for Language Recognition) es un metacompilador que opera sobre múltiples lenguajes de destino (Java, Python, C++, C#, Go, entre otros). A partir de una descripción gramatical construye parsers de tipo ALL(*) —una variante adaptativa de LL— e intérpretes, compiladores o traductores completos, incorporando acciones semánticas embebidas en el lenguaje de destino elegido. A diferencia de Bison, ANTLR genera por defecto el árbol de sintaxis (parse tree) sin necesidad de que el desarrollador lo construya manualmente en cada regla.

CRITERIO	BISON	ANTLR
ALGORITMO	<i>LALR(1) / GLR (bottom-up)</i>	<i>ALL(*), variante adaptativa de LL (top-down)</i>
LENGUAJES DE DESTINO	<i>C, C++</i>	<i>Java, Python, C++, C#, Go, JavaScript, entre otros</i>
CONSTRUCCIÓN DEL AST	<i>Manual, mediante acciones semánticas en cada regla</i>	<i>Automática; se recorre luego con visitors o listeners</i>
RECUPERACIÓN DE ERRORES	<i>Manual: requiere declarar el token error en la gramática</i>	<i>Automática por defecto (DefaultErrorStrategy), sobrescribible</i>

Gestión de errores

La literatura de compiladores agrupa las estrategias de recuperación de errores sintácticos en cuatro familias: modo pánico (descartar tokens hasta encontrar uno

sincronizador), recuperación a nivel de frase (sustituir, insertar o eliminar tokens puntuales en el punto del error), producciones de error (aumentar la gramática con reglas que reconocen construcciones erróneas comunes) y corrección global (buscar la secuencia de cambios de menor costo que hace válida la entrada, poco usada en la práctica por su costo computacional). Bison y ANTLR implementan variantes distintas de estas familias.

Bison: recuperación basada en el token error

Bison no recupera errores por defecto: si no se declara nada, ante un token inesperado el parser generado invoca la rutina `yerror` y detiene el análisis. La recuperación se activa declarando un token especial reservado, `error`, dentro de una producción (por ejemplo, `stmt: error ';'`). Cuando ocurre un error, Bison desapila la pila de análisis hasta encontrar un estado donde ese token pueda desplazarse, y luego descarta los tokens de entrada restantes hasta hallar uno que permita continuar (en el ejemplo, hasta el siguiente punto y coma). Esto corresponde a una variante de modo pánico guiada por la propia gramática: el alcance y el punto de sincronización dependen enteramente de las reglas de error que el desarrollador decida escribir.`

ANTLR: estrategia de recuperación automática

ANTLR incorpora por defecto una clase (`DefaultErrorStrategy`) que actúa sin que el desarrollador declare nada en la gramática. Ante un token que no coincide con lo esperado, intenta primero una inserción de un único token: si el símbolo en la segunda posición de anticipación (`lookahead`) sí coincide con lo esperado, asume que falta un token, lo genera de forma sintética y continúa. Si eso no aplica, intenta una eliminación de un único token: si el símbolo esperado aparece en la posición siguiente, descarta el token sobrante y continúa.

Cuando ninguna de las dos estrategias resuelve el conflicto, ANTLR recurre a un mecanismo equivalente al modo pánico: consume tokens de entrada hasta encontrar uno que pertenezca al conjunto de continuación (follow set) válido para la regla activa. Esta estrategia por defecto puede sobrescribirse implementando una clase de error propia cuando el lenguaje lo requiere.

Contraste

La diferencia práctica entre ambos generadores es el nivel de automatismo: en Bison la calidad de la recuperación depende directamente de cuántas producciones de error escriba el desarrollador, mientras que en ANTLR existe una estrategia de recuperación de propósito general activa desde el primer momento, aplicable a cualquier gramática sin anotación adicional. Ninguna de las dos aproximaciones repara el error de forma semánticamente correcta; ambas buscan que el análisis pueda continuar para reportar la mayor cantidad de errores posible en una sola pasada, en lugar de detenerse en el primero.

Referencias

- Árboles de análisis abstracto | PL 22/23. (s. f.). Recuperado de <https://ull-esit-gradoii-pl.github.io/temas/syntax-analysis/ast.html>
- Abstract syntax trees. (s. f.). Recuperado de <https://cs.ecu.edu/abrahamsonk/5220/spr16/Notes/Review/ast.html>
- parsingtheory. (s. f.). Recuperado de https://cs-lmu-edu.translate.goog/~ray/notes/parsingtheory/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
- Medeiros, S., & Mascarenhas, F. (2018). Syntax error recovery in parsing expression grammars. *In Proceedings Of SAC 2018: Symposium On Applied Computing*, 1195-1202. <https://doi.org/10.1145/3167132.3167261>
- Diekmann, L., & Tratt, L. (2018). Don't Panic! Better, Fewer, Syntax Errors for LR Parsers. *arXiv (Cornell University)*. <https://doi.org/10.4230/lipics.ecoop.2020.6>